BÀI THỰC HÀNH PHẦN KERNEL SUPPORT VECTOR MACHINE

Ví dụ 1: Chúng ta vẫn bắt đầu với một ví dụ trên dữ liệu nhân tạo. Trước hết ta tạo một bộ dữ liệu có 200 mẫu (N=200) phân đều vào 02 lớp – mỗi lớp 100 phần tử. Mỗi mẫu có 2 chiều (d=2). Dữ liệu sẽ được chúng ta tạo bởi make_circles trong thư viện sklearn.dataset sao cho đường biên giữa các lớp là không tuyến tính.

Tiếp theo sử dụng thư viện sklearn để minh họa phương pháp Kernel Vector Machine với các hàm nhân khác nhau và tiến hành thực hiện phân lớp trên bộ dữ liệu nói trên.

Đoạn lệnh tạo dữ liệu

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm
from matplotlib.backends.backend_pdf import PdfPages
from sklearn.datasets import make_circles

# Generate dataset and targets
X, Y = make_circles(n_samples = 200, noise = 0.02)

# visualizing data
plt.scatter(X[:, 0], X[:, 1], c = Y, marker = '.')
plt.show()
```

Đoạn lệnh gọi thư viện và thực hiện quá trình phân loại, sau đó vẽ ra đường biên để quan sát

```
fignum = 1

# fit the model
for kernel in ('linear','sigmoid', 'poly', 'rbf'):
    clf = svm.SVC(kernel=kernel, gamma=1, coef0 = 1)
    clf.fit(X, Y)
    with PdfPages(kernel + '3.pdf') as pdf:
        # plot the line, the points, and the nearest vectors to the plane
        fig, ax = plt.subplots()
        plt.figure(fignum, figsize=(5, 5))
        plt.clf()

        plt.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1], s=80,
                    facecolors='None')
        plt.plot(X[Y==0, 0], X[Y==0, 1], 'bs', markersize = 2)
        plt.plot(X[Y==1, 0], X[Y==1, 1], 'ro', markersize = 2)

        plt.axis('tight')
        x_min = -1.5
        x_max = 1.5
        y_min = -1.5
        y_max = 1.5

        XX, YY = np.mgrid[x_min:x_max:200j, y_min:y_max:200j]
        Z = clf.decision_function(np.c_[XX.ravel(), YY.ravel()])

        # Put the result into a color plot
        Z = Z.reshape(XX.shape)
        plt.figure(fignum, figsize=(5, 5))
        CS = plt.contourf(XX, YY, np.sign(Z), 200, cmap='jet', alpha = .2)
        plt.contour(XX, YY, Z, colors=['k', 'k', 'k'], linestyles=['--', '-', '--'],
                    levels=[-.5, 0, .5])
        plt.title(kernel, fontsize = 15)
        plt.xlim(x_min, x_max)
        plt.ylim(y_min, y_max)

        plt.xticks(())
```

```
        plt.yticks(())
        fignum = fignum + 1
        pdf.savefig()
plt.show()
```

Bài tập 1. Sử dụng thư viện để đưa ra độ chính xác Accuracy và ma trận nhầm lẫn Confusion Matrix theo mỗi phương pháp (Tham khảo code từ các bài trước).

Ví dụ 2. Trong ví dụ 2, chúng ta sử dụng dữ liệu kiểm tra chất lượng dự đoán liệu các vi mạch từ một nhà máy chế tạo có đảm bảo chất lượng hay không. Tệp dữ liệu có trong file ex2data2.txt đính kèm. Dữ liệu quan sát gồm 2 chiều, đầu ra Y có dạng 0-1.

Dưới đây là đoạn lệnh đọc dữ liệu và vẽ ra màn hình

```
import matplotlib.pyplot as plt
from sklearn import svm
from matplotlib.backends.backend_pdf import PdfPages

# visualizing data
mask = Y == 1
passed = plt.scatter(X[mask][0].values, X[mask][1].values)
failed = plt.scatter(X[~mask][0].values, X[~mask][1].values)
plt.xlabel('Microchip Test1')
plt.ylabel('Microchip Test2')
plt.legend((passed, failed), ('Passed', 'Failed'))
plt.show()
```

Tương tự Ví dụ 1, chúng ta sẽ sử dụng các nhân có trong thư viện sklearn để phân loại dữ liệu

```
fignum = 1

# fit the model
for kernel in ('linear','sigmoid', 'poly', 'rbf'):
    clf = svm.SVC(kernel=kernel, gamma=1, coef0 = 1)
    clf.fit(X, Y)
    with PdfPages(kernel + '3.pdf') as pdf:
        # plot the line, the points, and the nearest vectors to the plane
        fig, ax = plt.subplots()
        plt.figure(fignum, figsize=(5, 5))
        plt.clf()

        plt.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1], s=80,
                    facecolors='None')
        plt.plot(X[mask][0].values, X[mask][1].values, 'bs', markersize = 2)
        plt.plot(X[~mask][0].values, X[~mask][1].values, 'ro', markersize = 2)

        plt.axis('tight')
        x_min = -1.5
        x_max = 1.5
        y_min = -1.5
        y_max = 1.5

        XX, YY = np.mgrid[x_min:x_max:200j, y_min:y_max:200j]
        Z = clf.decision_function(np.c_[XX.ravel(), YY.ravel()])

        # Put the result into a color plot
        Z = Z.reshape(XX.shape)
        plt.figure(fignum, figsize=(5, 5))
        CS = plt.contourf(XX, YY, np.sign(Z), 200, cmap='jet', alpha = .2)
        plt.contour(XX, YY, Z, colors=['k', 'k', 'k'], linestyles=['--', '-', '--'],
```

```
                    levels=[-.5, 0, .5])
        plt.title(kernel, fontsize = 15)
        plt.xlim(x_min, x_max)
        plt.ylim(y_min, y_max)

        plt.xticks(())
        plt.yticks(())
        fignum = fignum + 1
        pdf.savefig()
plt.show()
```

Bài tập tự thực hành 1. Cho tệp dữ liệu đính kèm ARgender.mat, đoạn lệnh sau đây sẽ đọc tệp dữ liệu đó và chia làm hai phần dữ liệu Training và Test Data. Chú ý sửa lại đường dẫn đến file dữ liệu theo thiết lập của các bạn:

```
import scipy.io as sio
from sklearn.svm import SVC

A = sio.loadmat('D:\\Teach_n_Train\\Machine Learning\\code\\data\\ARgender.mat')
X_train = A['Y_train'].T
X_test = A['Y_test'].T
print(X_train.shape)
N = 700
y_train = A['label_train'].reshape(N)
y_test = A['label_test'].reshape(N)
```

Hãy tham khảo các ví dụ ở trên, sau đó thực hiện train với mô hình Kernel SVM - ứng với các nhân khác nhau – trên tập dữ liệu training.

Sau đó hãy thực hiện dự đoạn (cùng mô hình) cho tập dữ liệu Test và dùng thư viện để đánh giá độ chính xác của mỗi trường hợp.

Bài tập tự thực hành 2. Cho tệp dữ liệu đính kèm dataset.csv, đoạn lệnh sau đây sẽ đọc tệp dữ liệu đó và chia làm hai phần dữ liệu Training và Test Data. Chú ý sửa lại đường dẫn đến file dữ liệu theo thiết lập của các bạn:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset = pd.read_csv('D:\\Teach_n_Train\\Machine Learning\\code\\data\\dataset.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values

from sklearn.model_selection import train_test_split
XTrain, XTest, yTrain, yTest = train_test_split(X, y, test_size = 0.25, random_state = 0)

# Chuyển vị ma trận dữ liệu do ta đọc bằng Pandas => mỗi record là 1 dong
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
XTrain = sc.fit_transform(XTrain)
XTest = sc.transform(XTest)
```

Hãy thực hiện các yêu cầu như với **Bài tập tự thực hành** 1.

Bài tập tự thực hành 3. Hãy thực hiện các yêu cầu như hai bài tập tự thực hành 1 và 2 với bài toán phân loại tập dữ liệu ung thư vú. Dữ liệu này có sẵn trong sklearn.dataset, chúng ta có thể dùng đoạn lệnh sau để tải dữ liệu từ thư viện này và hiện thị các trường của dữ liệu để dễ theo dõi:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
plt.style.use('ggplot')

# Breast cancer dataset for classification
data = load_breast_cancer()
print (data.feature_names)
print (data.target_names)
```

Dữ liệu cũng có thể đọc từ tệp data.csv đính kèm. Chú ý đầu ra (Y) nguyên bản của dữ liệu là 02 lớp ['malignant' 'benign'] (text) do đó ta cần kỹ thuật để chuyển chúng về dạng 0, 1 (tham khảo các bài đã có).